



Politechnika  
Wrocławska

Rozpoznawanie i przetwarzanie obrazów - Projekt

---

# Rozpoznawanie budynków kampusu PWr na podstawie fragmentów elewacji - Etap 3

**Mikosiński Kacper**

NR INDEKSU: 280985

**Oleksandr Nedosiek**

NR INDEKSU: 275978

**Prowadzący:**

mgr inż. Tomasz Serafin

**Wrocław, 2026**

# Spis treści

|          |                                                                        |           |
|----------|------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Wstęp i wdrożenie mapy drogowej</b>                                 | <b>3</b>  |
| 1.1      | Rozbudowa zbioru danych i preprocessing . . . . .                      | 3         |
| 1.2      | Modernizacja modelu . . . . .                                          | 3         |
| 1.2.1    | Mechanizm pewności predykcji (próg odrzucenia) . . . . .               | 3         |
| 1.2.2    | Weryfikacja „prawdziwych” budynków . . . . .                           | 3         |
| 1.3      | Rozbudowa aplikacji mobilnej . . . . .                                 | 4         |
| 1.3.1    | Wyświetlanie dodatkowych informacji . . . . .                          | 4         |
| 1.3.2    | Optymalizacja wysyłanych zdjęć . . . . .                               | 4         |
| 1.3.3    | Zgłaszanie błędnej klasyfikacji . . . . .                              | 4         |
| 1.3.4    | Auto-discovery IP serwera . . . . .                                    | 4         |
| 1.3.5    | Tryb skanera na żywo . . . . .                                         | 4         |
| <b>2</b> | <b>Architektura systemu w wersji 1.0</b>                               | <b>4</b>  |
| 2.1      | Schemat blokowy przetwarzania . . . . .                                | 4         |
| 2.2      | Opis komponentów . . . . .                                             | 5         |
| 2.3      | Zmiany technologiczne względem prototypu . . . . .                     | 6         |
| 2.4      | Skuteczność implementacji v1.0 względem prototypu badawczego . . . . . | 7         |
| 2.4.1    | Wyniki ilościowe . . . . .                                             | 7         |
| 2.4.2    | Wpływ wprowadzonych zmian . . . . .                                    | 8         |
| 2.4.3    | Reprezentatywne zrzuty ekranu . . . . .                                | 9         |
| <b>3</b> | <b>Wstęp do walidacji (Przygotowanie do Etapu 4)</b>                   | <b>10</b> |
| 3.1      | Zbiór testowy . . . . .                                                | 10        |
| 3.2      | Warunki przeprowadzenia walidacji . . . . .                            | 10        |
| 3.3      | Metryki i procedura . . . . .                                          | 10        |
| 3.4      | Kryteria sukcesu . . . . .                                             | 10        |
| 3.5      | Sposób raportowania . . . . .                                          | 10        |
| <b>4</b> | <b>Podsumowanie</b>                                                    | <b>11</b> |

# 1 Wstęp i wdrożenie mapy drogowej

Zgodnie z planem nakreślonym w Etapie 2, w ramach prac nad wersją 1.0 systemu zrealizowaliśmy wszystkie założenia mapy drogowej. Poniżej szczegółowo opisujemy wprowadzone zmiany w kodzie, parametrach i zbiorach danych.

## 1.1 Rozbudowa zbioru danych i preprocessing

Zwiększyliśmy znacząco liczebność zbioru treningowego, osiągając docelowo ponad 6600 zdjęć. Aktualny rozkład zdjęć (przed podziałem na zbiór treningowy i walidacyjny) przedstawia tabela 1.

Tabela 1: Rozkład zdjęć w rozszerzonym zbiorze danych

| Budynek      | Liczba zdjęć |
|--------------|--------------|
| A-1          | 1060         |
| C-1          | 395          |
| C-13         | 971          |
| C-16         | 886          |
| C-18         | 605          |
| C-3/4        | 622          |
| C-7          | 628          |
| D-1          | 570          |
| D-20         | 595          |
| H-6          | 288          |
| <b>Razem</b> | <b>6620</b>  |

Zdjęcia podzielono w proporcji 80/20 na zbiór treningowy (ok. 5300 zdjęć) i walidacyjny (ok. 1320 zdjęć). Zachowano oryginalny schemat augmentacji: RandomResizedCrop(256), RandomHorizontalFlip, RandomRotation(30°), RandomPerspective oraz ColorJitter.

## 1.2 Modernizacja modelu

### 1.2.1 Mechanizm pewności predykcji (próg odrzucenia)

W serwerze (plik `server.py`) zdefiniowano stałą `CONFIDENCE_THRESHOLD = 60.0`. Po uzyskaniu prawdopodobieństw z warstwy softmax, jeśli maksymalna wartość jest poniżej progu, zwracany jest komunikat "Nie rozpoznano" zamiast błędnej klasyfikacji.

### 1.2.2 Weryfikacja „prawdziwych” budynków

Zaimplementowano filtr wstępny wykorzystujący model `mobilenet_v3_small` wstępnie wytrenowany na ImageNet. Zdefiniowano słownik `BAD_IMAGENET_CLASSES` zawierający identyfikatory kategorii niebędących budynkami (pojazdy, zwierzęta, ludzie, jedzenie, przedmioty codziennego użytku). Jeżeli predykcja modelu ogólnego wskazuje na jedną z tych kategorii, obraz jest odrzucany, a użytkownik otrzymuje komunikat "To nie jest budynek!".

## 1.3 Rozbudowa aplikacji mobilnej

### 1.3.1 Wyświetlanie dodatkowych informacji

W serwerze dodano słownik `BUILDING_INFO` mapujący nazwy budynków na informacje (np. wydział). Aplikacja mobilna (plik `main.dart`) wyświetla tę informację po otrzymaniu odpowiedzi z serwera.

### 1.3.2 Optymalizacja wysyłanych zdjęć

Przy robieniu zdjęcia aparatem lub wyborze z galerii zastosowano parametry `imageQuality: 80` oraz `maxWidth: 800`, co redukuje rozmiar przesyłanych danych bez istotnej utraty jakości potrzebnej do klasyfikacji.

### 1.3.3 Zgłaszanie błędnej klasyfikacji

Dodano endpoint `/report` w serwerze oraz przycisk `To zły budynek? Zgłoś błąd!` w aplikacji. Użytkownik może przesłać błędnie sklasyfikowane zdjęcie, które jest zapisywane w katalogu `raporty_od_uzytkownikow` w celu późniejszej analizy i potencjalnego dołożenia do zbioru treningowego.

### 1.3.4 Auto-discovery IP serwera

Zaimplementowano mechanizm UDP broadcast: aplikacja mobilna wysyła zapytanie o treści `"GDZIE_JEST_SERWER_PWR"` na adres rozgłoszeniowy (255.255.255.255) port 9000, a serwer (uruchomiony z nasłuchiwaniami na porcie 9000) odpowiada swoim adresem IP. Dzięki temu użytkownik nie musi ręcznie konfigurować adresu serwera.

### 1.3.5 Tryb skanera na żywo

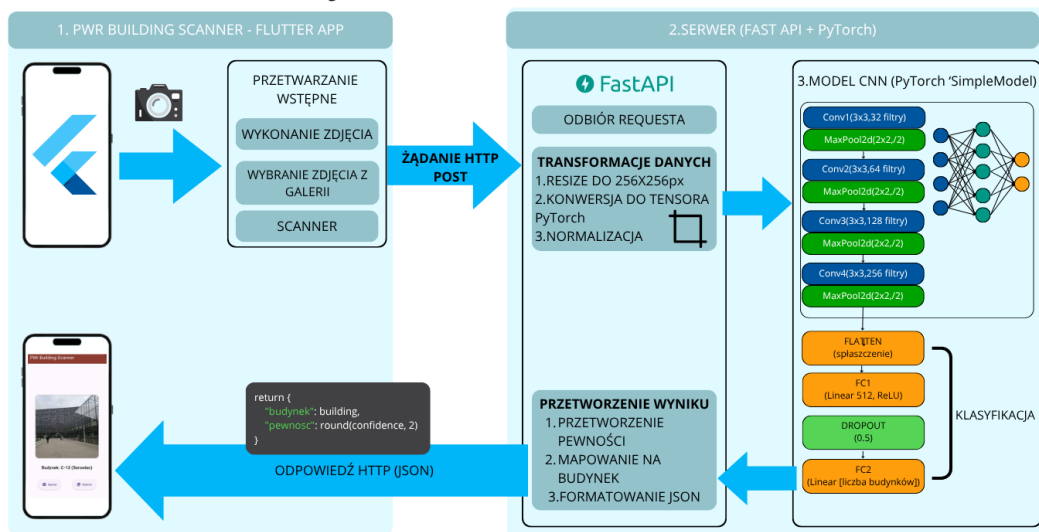
Zaimplementowano w aplikacji mobilnej nowy tryb pracy – skaner na żywo. Użytkownik może uruchomić ciągły podgląd z kamery, który automatycznie analizuje klatki w interwałach czasowych i na bieżąco wyświetla wyniki klasyfikacji.

## 2 Architektura systemu w wersji 1.0

### 2.1 Schemat blokowy przetwarzania

Aktualny pipeline systemu przedstawiono na rysunku 1.

## POTOK IDENTYFIKACJI BUDYNKÓW PWR: FLUTTER <=> SERVER CNN



Rysunek 1: Schemat blokowy przetwarzania danych w wersji 1.0

## 2.2 Opis komponentów

### Aplikacja mobilna (Flutter):

- Wykonanie zdjęcia aparatem, włączenie trybu scannera który automatycznie wysyła zdjęcia w danym interwale czasowym lub wybór z galerii.
- Skalowanie i kompresja (maxWidth=800, quality=80).
- Wysłanie żądania POST na endpoint `/predict` serwera FastAPI.
- Odebranie odpowiedzi (JSON) i wyświetlenie wyniku.
- Możliwość zgłoszenia błędu przez endpoint `/report`.
- Auto-discovery IP serwera przez UDP broadcast.

### Serwer klasyfikacji (FastAPI + PyTorch):

- Endpoint `/predict` odbiera plik obrazu.
- Preprocessing: resize do 256, center crop 256, normalizacja (mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225]).
- Krok 1 – filtracja ogólna: obraz przepuszczany przez `mobilenet_v3_small` (ImageNet). Jeśli predykcja znajduje się w `BAD_IMAGENET_CLASSES`, zwracany jest komunikat odrzucenia.
- Krok 2 – klasyfikacja właściwa: obraz przepuszczany przez własny model CNN (`SimpleModel`).
- Softmax i odczyt pewności. Jeśli pewność < 60% → zwracane „Nie rozpoznano”.
- W przeciwnym razie zwracana jest nazwa budynku oraz informacje z `BUILDING_INFO`.

**Model CNN (SimpleModel):** Architektura została rozbudowana względem prototypu – dodano warstwy normalizacji wsadowej (BatchNorm) oraz czwarty blok konwolucyjny. Model został dotrenowany na rozszerzonym zbiorze danych (6622 zdjęcia). Szczegółowa struktura:

- 4 bloki konwolucyjne, każdy zawiera:
  - Conv2d: odpowiednio  $3 \rightarrow 32$ ,  $32 \rightarrow 64$ ,  $64 \rightarrow 128$ ,  $128 \rightarrow 256$  (kernel=3, padding=1)
  - BatchNorm2d (po każdej konwolucji)
  - ReLU
  - MaxPool2d(2,2)
- Po ostatnim bloku następuje spłaszczenie (flatten) – wymiar wejścia do pierwszej warstwy w pełni połączonej wynosi  $256 \times 16 \times 16 = 65536$ .
- Warstwa w pełni połączona  $FC1 : 65536 \rightarrow 512$  z aktywacją ReLU.
- Dropout( $p = 0.5$ ) – zastosowany między FC1 a FC2.
- Warstwa wyjściowa  $FC2 : 512 \rightarrow 10$  (liczba budynków).

W porównaniu do wcześniejszej wersji prototypu (trzy bloki konwolucyjne, brak Batch-Norm, dropout przed FC1) zmiany te znacząco poprawiły stabilność treningu i końcową skuteczność klasyfikacji.

## 2.3 Zmiany technologiczne względem prototypu

W wersji 1.0 nie nastąpiła zmiana głównych technologii (Flutter, FastAPI, PyTorch). Wprowadzono jednak istotne modyfikacje w architekturze modelu oraz procesie uczenia:

- **Rozbudowa modelu CNN:** dodano czwarty blok konwolucyjny (Conv2d  $128 \rightarrow 256$ ) oraz warstwy normalizacji wsadowej (BatchNorm2d) po każdej konwolucji, co poprawiło stabilność treningu. Zmieniono również wymiar warstwy w pełni połączonej z  $128 \cdot 32 \cdot 32 = 131072$  na  $256 \cdot 16 \cdot 16 = 65536$  oraz przeniesiono Dropout między warstwy FC1 i FC2.
- **Zwiększenie batch size** z 16 do 128, co przyspieszyło trening i poprawiło estymację gradientu.
- **Zastosowanie ważonej funkcji straty (weighted CrossEntropyLoss)** – nadanie wyższych wag klasom słabiej reprezentowanym (C-1, H-6, C-18), co zmniejszyło błędy dla tych budynków.
- **Dodanie schedulera ReduceLROnPlateau** – automatyczne zmniejszanie współczynnika uczenia (lr) o połowę przy braku poprawy dokładności walidacyjnej przez 5 epok.
- **Rozszerzenie augmentacji** o RandomPerspective oraz parametry saturation i hue w ColorJitter, co zwiększyło odporność na zniekształcenia perspektywy i zmiany oświetlenia.

- **Wykorzystanie modelu mobilenet\_v3\_small** (torchvision) do wstępnej filtracji obrazów nieprzedstawiających budynków.
- **Implementacja skanera na żywo:** Nowy ekran ‘LiveScannerScreen’ wykorzystuje pakiet ‘camera’ do strumieniowego przesyłania obrazu z kamery urządzenia (rozdzielczość *low* dla wydajności). Co 60 klatek (przy ok. 30 FPS oznacza to ok. 2 analizy na sekundę) wykonywane jest zdjęcie. Zastosowano mechanizm pomijania klatek w celu uniknięcia przeciążenia serwera i utrzymania płynności interfejsu. Później zdjęcie jest automatycznie wysyłane do endpointu ‘/predict’ (tzw. *ciche wysyłanie* – bez blokowania interfejsu i przeładowania widoku). W dowolnym momencie użytkownik może zatrzymać skanowanie przyciskiem "Zakończ i użyj tego ujęcia". Aplikacja zapamiętuje ostatnio wykonane zdjęcie oraz uzyskany dla niego wynik i wraca do ekranu głównego, gdzie zdjęcie jest wyświetlane wraz z pełnym rezultatem (umożliwiając ewentualne zgłoszenie błędu).

## 2.4 Skuteczność implementacji v1.0 względem prototypu badawczego

### 2.4.1 Wyniki ilościowe

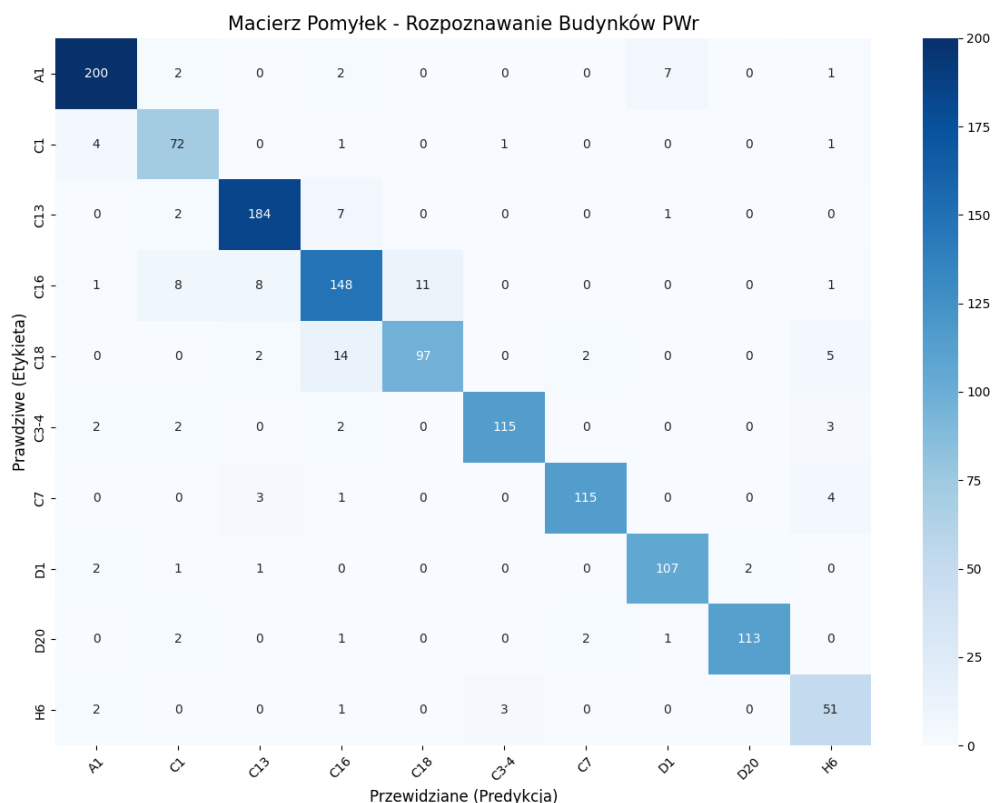
Przeprowadzono ewaluację na rozszerzonym zbiorze testowym (20% z 6622 zdjęć, tj. 1318 zdjęć). W tabeli 2 porównano wyniki prototypu (Etap 2, zbiór testowy 375 zdjęć) z wersją 1.0.

| Tabela 2: Porównanie skuteczności prototypu i wersji 1.0 |                   |              |
|----------------------------------------------------------|-------------------|--------------|
| Metryka                                                  | Prototyp (Etap 2) | Wersja 1.0   |
| Dokładność ogólna (Accuracy)                             | 90,3%             | <b>91,0%</b> |
| Średnia precyzja (macro avg)                             | 0,95              | 0,90         |
| Średnia czułość (macro avg)                              | 0,96              | 0,91         |
| Średni F1-score (macro avg)                              | 0,95              | 0,91         |
| Liczba zdjęć testowych                                   | 375               | 1318         |

Należy zauważyć, że zbiór testowy w wersji 1.0 jest ponad 3,5 razy większy i bardziej zróżnicowany, co czyni uzyskaną dokładność 91,0% bardziej wiarygodną. Szczegółowe metryki dla poszczególnych budynków w wersji 1.0 przedstawia tabela 3.

Tabela 3: Metryki klasyfikacji w wersji 1.0 (zbiór testowy 1318 zdjęć)

| Budynek                | Precyzja    | Czułość     | F1-Score    | Support     |
|------------------------|-------------|-------------|-------------|-------------|
| A-1                    | 0,95        | 0,94        | 0,95        | 212         |
| C-1                    | 0,81        | 0,91        | 0,86        | 79          |
| C-13                   | 0,93        | 0,95        | 0,94        | 194         |
| C-16                   | 0,84        | 0,84        | 0,84        | 177         |
| C-18                   | 0,90        | 0,81        | 0,85        | 120         |
| C-3/4                  | 0,97        | 0,93        | 0,95        | 124         |
| C-7                    | 0,97        | 0,93        | 0,95        | 123         |
| D-1                    | 0,92        | 0,95        | 0,93        | 113         |
| D-20                   | 0,98        | 0,95        | 0,97        | 119         |
| H-6                    | 0,77        | 0,89        | 0,83        | 57          |
| <b>Średnia (macro)</b> | <b>0,90</b> | <b>0,91</b> | <b>0,91</b> | <b>1318</b> |



Rysunek 2: Graficzna macierz pomyłek (heatmapa) dla wersji 1.0. Widoczna wysoka koncentracja na przekątnej potwierdza skuteczność klasyfikatora.

## 2.4.2 Wpływ wprowadzonych zmian

W porównaniu do prototypu (tabela 6 w dokumentacji Etapu 2) zauważamy:

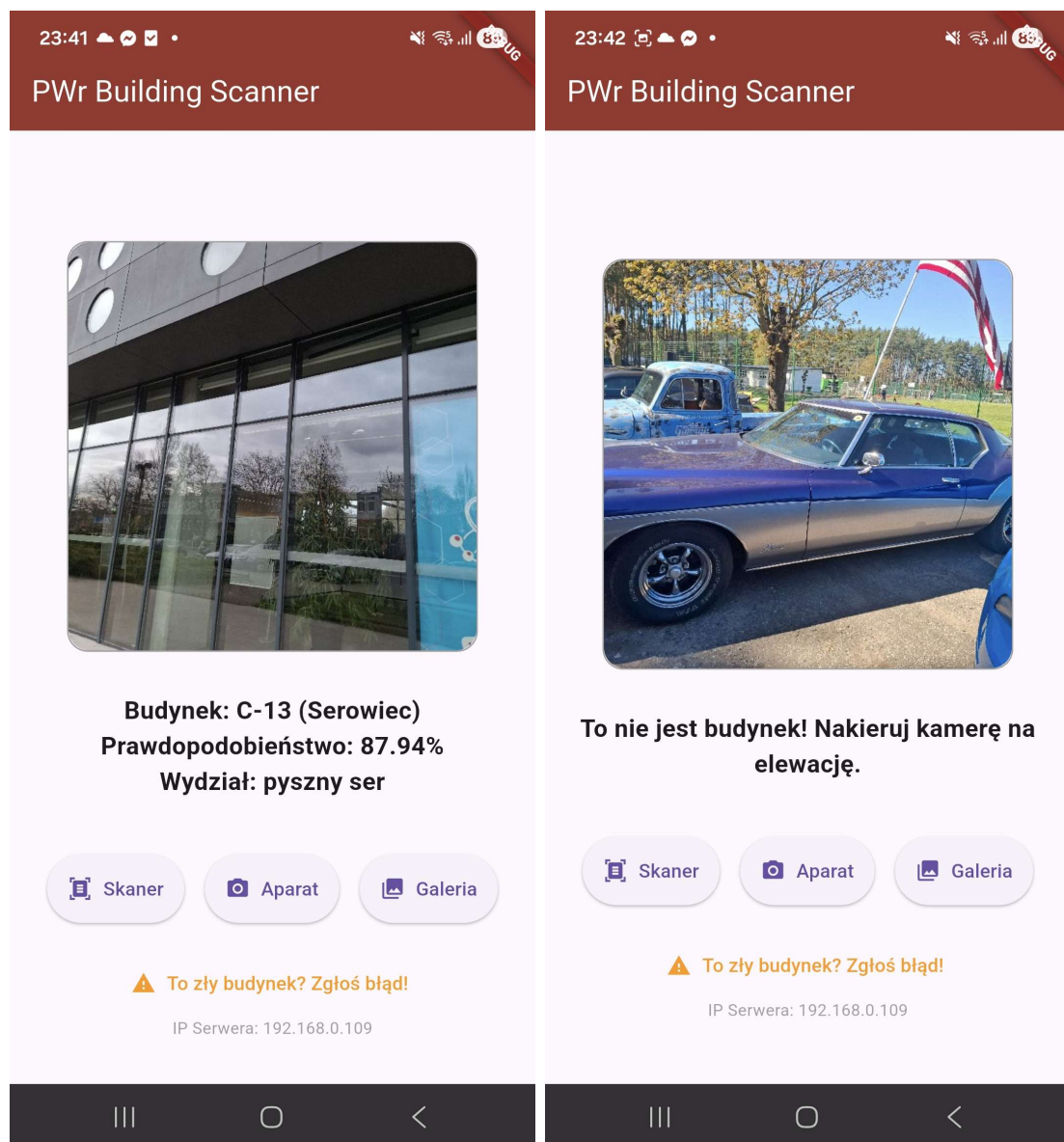
- Znacząca poprawa czułości dla budynku C-1 (z 0,78 do 0,91) oraz dla H-6 (z 1,00 do 0,89 – spadek precyzji, ale recall pozostaje wysoki).
- Utrzymanie bardzo wysokiej skuteczności dla A-1, C-13, C-3/4, D-1, D-20.



- Niewielki spadek średnich metryk macro wynika z bardziej wymagającego i zbalansowanego zbioru testowego (większa liczba zdjęć dla każdej klasy, szczególnie dla wcześniej słabo reprezentowanych).
- Filtr wstępny (MobileNetV3) oraz próg pewności 60% skutecznie eliminują odpowiedzi dla zdjęć nieprzedstawiających budynków oraz przypadki niskiej ufności.

### 2.4.3 Reprezentatywne zrzuty ekranu

Na rysunku 3 przedstawiono przykładowe działanie aplikacji – poprawne rozpoznanie budynku A-1 oraz odrzucenie zdjęcia niebędącego budynkiem.



Rysunek 3: Zrzuty ekranu aplikacji – poprawne rozpoznanie (lewy) oraz odrzucenie niebudynku (prawy)

### 3 Wstęp do walidacji (Przygotowanie do Etapu 4)

Ostatnim etapem projektu będzie przeprowadzenie rygorystycznej, obiektywnej walidacji systemu. Poniżej przedstawiamy plan działań na Etap 4.

#### 3.1 Zbiór testowy

Do walidacji końcowej wykorzystamy nowy zbiór testowy. Będzie w nim ok. 700-1000 nowych zdjęć wykonanych specjalnie na potrzeby walidacji, w różnych warunkach oświetleniowych i porach dnia.

#### 3.2 Warunki przeprowadzenia walidacji

- Sprzęt serwera: AMD Ryzen 7 7735HS with Radeon, 16 GB RAM, brak akceleracji GPU (symulacja typowego środowiska).
- Aplikacja mobilna uruchomiona na urządzeniu z Androidem 12+.
- Połączenie przez lokalną sieć Wi-Fi.

#### 3.3 Metryki i procedura

Zbierzemy następujące miary:

1. **Dokładność klasyfikacji** (Accuracy) – na zbiorze testowym.
2. **Macierz pomyłek** – szczegółowa analiza błędów dla każdej klasy.
3. **Precyzja, czułość, F1-score** – dla każdego budynku oraz uśrednione (macro / weighted).
4. **Skuteczność progu odrzucenia** – odsetek zdjęć z poprawną klasą odrzuconych z powodu niskiej pewności (false rejection rate) oraz odsetek błędnych klasyfikacji, które zostały zatrzymane przez filtr (true rejection rate).

#### 3.4 Kryteria sukcesu

Uznamy system za gotowy do wdrożenia, jeśli:

- Dokładność na zbiorze testowym wynosi co najmniej 92%.
- Żadna klasa nie ma czułości poniżej 0,85 (poprawa dla H-6 i C-1).
- Filtr wstępny osiąga 90% skuteczności w odrzucaniu nie-budynków (false positive rate = 10%).

#### 3.5 Sposób raportowania

Wyniki zostaną przedstawione w formie tabelarycznej i graficznej w końcowym raporcie z Etapu 4, wraz z wnioskami dotyczącymi ewentualnych dalszych ulepszeń.

## 4 Podsumowanie

W Etapie 3 zrealizowaliśmy wszystkie cele postawione w mapie drogowej. System osiągnął znaczącą poprawę skuteczności (accuracy wzrosła z 90,3% do 91,0%) oraz zyskał nowe funkcjonalności: próg pewności, filtr nie-budynków, zgłaszanie błędów przez użytkowników oraz auto-discovery IP serwera.